

Applying score-following tools for expressive-timing visualization

Jairus Joseph-Sioeli Rhoades
Jairus.Rhoades@yale.edu

Scott Eric Petersen
Scott.Petersen@yale.edu

*A Senior Thesis as a partial fulfillment of requirements
for the Bachelor of Arts in Computing in the Arts*

Department of Computer Science
Yale University
Dec 11, 2025

Acknowledgements

I want to express my deepest gratitude to the one person who could have had enough patience and grace to have dealt with me this semester, my thesis advisor, professor, and Director of Undergraduate of Studies, Scott Petersen. He, along with my advisor for the MUSI4996 component of this thesis, Konrad Kaczmarek, provided me so much advice and expertise on narrowing of the scope of this project so that it was a viable pursuit that would enable my graduation. It very much may not seem like it, but your inspiration as a musical figure and mentor has motivated me in so many ways to keep on pursuing music professionally and apply to graduate school music programs while making sure I bring these studies into the practice room.

Thank you to Sohee Park, professor and manager of the Senior Project class (CPSC4900), for keeping me accountable and inspiring humanity in my approach to assembling this project.

To Dean Cruz (D-Twista) for showing me the capabilities of TouchDesigner through his many sets at Plush and CafeNine.

To Ron Cheng for inspiring me with integrity and accountability, and for being one of the biggest role-models in computer science and academics here at Yale.

To Kristjan for bringing out the best of a study break.

To my beautiful housemates, Joy, Josh, George, and Phoebe, for being there with me in the living room working together. Every. Single. Night.

To Okole Puka Hui for living on my iMessage and for always encouraging me to keep on going.

To Emma Slagle for being here with me on this long journey from 6th grade at Punahou to being fellow CPAR-ians at Yale (of course, also for accountability on this project). Can you believe we're here?!

Lastly, to my parents for affirming me in my path to pursue music and computer science with stories of Grandpa. Thank you Mom for always checking in on me with the phone calls and Dad for the (free) Amazon stand up desk that the majority of this thesis was completed on. The House loves the desk as well.

Contents

1	Introduction	5
2	Background	7
2.1	Expressive Timing in Musical Performance	7
2.2	Scorefollowing as a Performance Tool	7
2.3	Visualization of Musical Structure and Performance Data	8
3	Methodology	9
3.1	Matchmaker Library Configuration	9
3.2	Analyze_warp.py: features and functionality	10
3.3	Event characterization	10
3.4	TouchDesigner / OSC in Analyze_warp.py	12
4	Results	13
4.1	Reconciling Playback Timing With Frame Output: Discovering What “Frame Rate” Actually Meant	13
4.2	Interpreting Early Behavior in tempo_ratio and delta_ratio	14
4.3	A Qualitative Description of Current TouchDesigner graphics	16
5	Related Work	19
6	Conclusions	21
7	Future Work	23
A	Appendix Title	26
A.1	Soundfont Details + Additional Assets	26
A.2	Matchmaker Sound Features	26
A.3	TouchDesigner	27

Applying score-following tools for expressive-timing visualization

Jairus Joseph-Sioeli Rhoades

Abstract

`analyze_warp.py` is used to examine the real-time alignment output of the previously implemented Matchmaker score-following library [1] and to expose internal per-frame metrics in an approach for visualization through a TouchDesigner project. The script extracts per-frame metrics including changes in score and performance indices, estimated tempo ratios, and event markers for clamping and score jumps. These outputs are examined across different sampling and hop-length configurations to document how Matchmaker advances, freezes, and re-anchors during alignment. An OSC interface transmits the per-frame stream to TouchDesigner, where the metrics drive a generative visualization. The resulting system illustrates the characteristic behaviors of Matchmaker's alignment process in real time, including sensitivity to frame-rate settings, intermittent periods of clamping, and sudden re-anchoring events. Smoothing and simple hysteresis methods are applied within TouchDesigner to stabilize rapidly changing signals for visual use. The work provides a technical account of how the alignment metrics behave when surfaced directly to an external visualization environment.

1 Introduction

Expressive timing is one of the defining characteristics of musical performance, shaping the perception of phrasing, tension, and emotional contour. Far from being incidental variation, expressive timing encodes interpretive decisions and carries information that listeners rely on to understand structure, intention, and style. Applications of expressive-timing analysis span performance pedagogy, comparative interpretation, musicology, etc. yet these approaches typically require detailed score annotations, sophisticated systems (often those using machine learning models, which introduce higher latency and cost). This creates a practical barrier for performers and researchers who may seek to study expressive timing. The emergence of accurate score-following systems creates the possibility of a more lightweight, accessible method for characterizing expressive behavior: if alignment metrics can be interpreted appropriately, score following can serve as a computational lens on expressive timing at far lower cost.

Real-time score following has matured significantly while still being a relatively dispersed research field. Systems developed at IRCAM such as Antescofo [2] demonstrate the feasibility of following a live performer with enough precision to coordinate electronics in performance. Park et al.’s development of the Matchmaker library [1] extends this capability into an open-source Python framework, enabling researchers to observe alignment behavior directly, modify processing components, and interface with external systems. While score following is often evaluated primarily on accuracy or latency, the internal alignment metrics themselves encode meaningful information about how a performer deviates from the score. These metrics, if understood properly, may offer a novel pathway to expressive-timing analysis without the need for dedicated annotation datasets or manual transcription.

This thesis asks three guiding research questions: (1) How can we meaningfully characterize expressive timing in performance using the low-level alignment metrics produced by a real-time score follower? (2) What metrics best capture the nuanced ways performers deviate from notated time? (3) How might these rapidly changing alignment signals be represented visually so that the dynamics of the alignment become perceptible in real time?

This work sits at the intersection of expressive-performance modeling, real-time score following, and visualization. Whereas expressive-timing studies often assume clean ground-truth alignments derived offline [3, 4], score-following systems instead infer alignment frame by frame under performance variability. Visualizations such as those developed by Gasser [5] illustrate expressive features elegantly but do not operate in real time. The present work bridges these domains by examining Matchmaker’s low-level alignment behavior and mapping that behavior into a generative visualization system capable of responding to performance input.

The contributions of this thesis could benefit these many strands. First, it provides a detailed account of the real-time metrics produced by Matchmaker, including delta measures, tempo estimates, and event states such as clamping and score jumps. Second, it identifies how these metrics behave under changes in frame rate, hop length, and performance input, documenting sources of ambiguity and nonlinear behavior. Third, it introduces a workflow that streams per-frame metrics from `analyze_warp.py` to TouchDesigner through OSC, enabling real-time visual expression of alignment dynamics.

Methodologically, the project provides simply one python file written to sit within a working matchmaker library alongside one TouchDesigner project. Matchmaker's alignment stream is extracted, structured, and supplemented with event logic before being transmitted to a TouchDesigner network comprising feedback, color, displacement, and noise operators. This environment responds directly to alignment changes, creating a visual surface that reflects clamping periods, leading behavior, and re-anchoring events in real time.

The main results show that Matchmaker's alignment metrics behave consistently with its underlying algorithm, yet they present interpretive challenges. Clamping of the score index during ambiguous passages and sudden re-anchoring when salient features reappear generate sharp discontinuities in delta measures and tempo estimates. These behaviors are not errors but features of the alignment process, and they must be smoothed, aggregated, or otherwise contextualized for meaningful use in real-time visualization. Nevertheless, when these signals are stabilized both within the codebase and the TouchDesigner project, the visualization pipeline expresses alignment events clearly alongside playback, making aspects of expressive timing newly accessible and visible in a clarity that onlookers at the CPSC4900 poster fair can attest to reinforcing their perceptions of expressive timing.

Overall, this project demonstrates how real-time score following can function as flexible tool for expressive-timing analysis and visualization, offering a way to observe timing behavior computationally without the resource burden of machine learning. In doing so, it contributes both to the technical understanding of Matchmaker's alignment behavior and to the broader goal of making expressive-performance analysis more accessible to performers, researchers, and artists.

2 Background

2.1 Expressive Timing in Musical Performance

Expressive timing has long been recognized as a core dimension of musical performance, shaping how phrasing, tension, and structural cues are perceived not only by performers but by listeners more broadly. It is perceived not just in music but also in art and language, and can even reveal to listeners and performers organization and hierarchy in the musical flow[6]. Studies in expressive-performance modeling have shown that timing deviations from the notated score are often systematic rather than incidental, reflecting interpretive decisions shaped by harmonic structure, phrase boundaries, and stylistic conventions [3, 4, 7]. Machine-learning approaches have further demonstrated that expressive behavior can be represented through tempo curves or timing trajectories learned from recorded performances, underscoring the extent to which timing contributes to musical meaning. Because expressive timing manifests at both micro- and macro-timescales, computational tools that track or represent timing deviations provide valuable insight into performer intention and perceptual experience. Personally, as a performer, it has always been challenging gaining an objective ear that can listen to my decisions in timing in the same way I do to a recording of someone else playing.

2.2 Scorefollowing as a Performance Tool

Score following, originally introduced as a method for synchronizing symbolic and audio representations of music, has evolved into an essential component of live computer-music systems. Early alignment studies, such as the work of Park et al. [1], demonstrated how audio–score alignment could support real-time tracking of a performer’s position within a notated work. Subsequent systems expanded on these ideas by incorporating probabilistic models and dynamic programming techniques to make robust predictions under expressive timing variation.

Among these systems, Antescofo [2] represents a major step in using score following as a practical performance technology. Designed for mixed and electro-acoustic works, Antescofo aligns live performers with symbolic scores in real time and schedules electronic events accordingly, enabling precise interaction between human musicians and computational processes. Across such systems, a central challenge remains: alignment must adapt to expressive variation while providing musically usable real-time signals. Modern frameworks such as Matchmaker continue this lineage remarkably by offering flexible, low-latency alignment tools all consolidated into one python library with a plethora of features and configurations that could allow further development in many

directions.

2.3 Visualization of Musical Structure and Performance Data

Visual representations of musical timing provide an additional lens through which performance processes can be understood. Prior work on visualizing expressive performance has explored how timing deviations, articulation, or phrasing can be mapped to shape, motion, or color in order to foreground interpretive structure [5], albeit post-performance. In parallel, MIR research has used cost matrices and warp-path diagrams to visualize alignment behavior and to diagnose how algorithms respond to ambiguous or expressive passages [8]. These two lines of work—expressive-performance visualization and alignment-diagnostic visualization—rarely intersect, but together offer conceptual foundations for mapping timing data into visual forms.

The present project situates itself at this intersection by exposing Matchmaker’s real-time alignment efficient metrics to a TouchDesigner environment via OSC, allowing the alignment process itself to become a visual process. Rather than visualizing precomputed expressive features, the system visualizes the real-time inference process, producing a space where computational alignment and expressive behavior become visible simultaneously and open to as much interpretational versatility as if it was its own instrument in such a performance.

3 Methodology

This project is made up of two components: a python codebase and a touchdesigner project, both of which run on the same machine (see A for running on a separate machine)

3.1 Matchmaker Library Configuration

Analyze_warp.py is the only file produced in which this project that interacts with an existing and functional Matchmaker v0.2.1 library directly as of December 2025. The library's features include several score-alignment algorithms that enable flexible audio/MIDI score-following across backends (while notable that a few alignment methods support only either MIDI, audio performance files, or real-time audio streaming) post-run evaluation hooks that can easily be connected to output files, and configurable playback, all while offering a rich set of features on the audio analysis stage, such as Chroma features, Mel-frequency cepstral coefficients, mel-spectrograms, and log-spectral features. The following algorithms within the Matchmaker library are ready for use: `arzt`, `dixon`, `hmm`, `pthmm`

Analyze_warp.py is built using Matchmaker's highest-accuracy algorithm, `arzt`, a dynamic, on-line time warping alignment algorithm built upon using the work done by Brazier, Widmer (2020), and Arzt et al.(2010). All alignment methods function through processing each MIDI (.midi) or musicXML (.musicxml) file into a feature array, which is batched into discrete step-like frames. When a .musicXML score file is used, an online synthetic rendering of the score (rendering required the use of a soundfont, which was selected for the scope of testing in this project to be a Steinway Model B Piano grabbed from Polyphone, an online soundfont database. More on soundfont default selection in appendix A), but for the selected as a Steinway Model B Piano grabbed online) is ran and compared against a recording and preliminary adjusts the tempo and thusly total frames. Upon the actual running of Matchmaker with either a pre-recorded audio file or to live audio, `mm.run()`, the main function capsulated in the `matchmaker.py` file, runs and steps through each frame in the pre-processed score in a queue that compares against small sample frames of the performance audio. Each hop is extracted by optional features (such as chroma/mel/etc., see appendix A), pushed into the queue, and the score follower matches it to a reference frame, updating the warping path (`ref_idx`, `perf_idx`) in real time.

Beyond the (n)ASAP materials in the original Matchmaker paper, I introduced custom Brahms excerpts (first seven bars), supplying both a MusicXML score (`matchmaker/assets/Brahms_I_XML.xml`) and a matching recording (`matchmaker/assets/Brahms_I_test.wav`), plus a 2x's speed recording (`matchmaker/assets/`

Brahms_fast.m4a) used for debugging. All Brahms alignment and evaluation runs therefore reflect an external, user-provided dataset rather than the canonical (n)ASAP set.

3.2 Analyze_warp.py: features and functionality

Analyze_warp.py acts as the inspection layer of said warp paths: it instantiates Matchmaker when in `--live` mode, streams frames to terminal in a for loop, and records how the alignment evolves. A simple command parser exposes all implemented controls (score file, performance file, input type, alignment method, frame rate, save path to tsv file, playback, OSC). For offline inspection (i.e. one not using `mm.run()`) used in debugging output, a "non-live" mode was accounted for that will expect an exist warp-path TSV made previously. Live runs initiate by computing the path, printing per-frame diagnostics and optionally writes the enriched path to TSV for later analysis. Wiring Matchmaker's diagnostic suite of features, I also flushed out post-processing graph plotting to generate the similarity matrix with the computed warp path overlaid. This visualization was produced automatically at the end of each alignment run and served as a key tool for analyzing how the follower interpreted the score-performance feature space. At each frame, the script reads the current (`ref_idx`, `input_idx`) from the score follower and derives the following:

1. `ref_idx/frame_rate`, the time from the beginning of the reading of the score.
2. `perf_idx/frame_rate`, the time from the beginning of the first frame of the performance synced to the first frame of the score (a matchmaker-level feature).
3. Δ_{ref} , Δ_{perf} , changes in the reference or performance indices.
4. `delta_ratio` and `tempo_ratio` where `tempo_ratio` is defined only when the score advances, and `delta_ratio` functioning as stand-in during these freezing events.
5. `dt_score`, computed as $\Delta_{\text{ref}} / \text{frame_rate}$ an inter-frame spacing metric that is useful in looking at score jumping events, which occur frequently (see results 4). If it is the first frame, it is set to `nan`.

3.3 Event characterization

From the metrics computed above, `analyze_warp.py` also detects several events and computed them for more output processing. It flags `perf_leading` runs when the performance advances while the score is clamped ($\Delta_{\text{ref}} = 0$, $\Delta_{\text{perf}} > 0$) and "score_jump" runs when the score advances by ≥ 5 frames; contiguous runs are summarized at the end of each run in the analysis portion of a run. These per-frame metrics are printed in real time (with `score_jump` events being recorded in the output frame after its completion.)

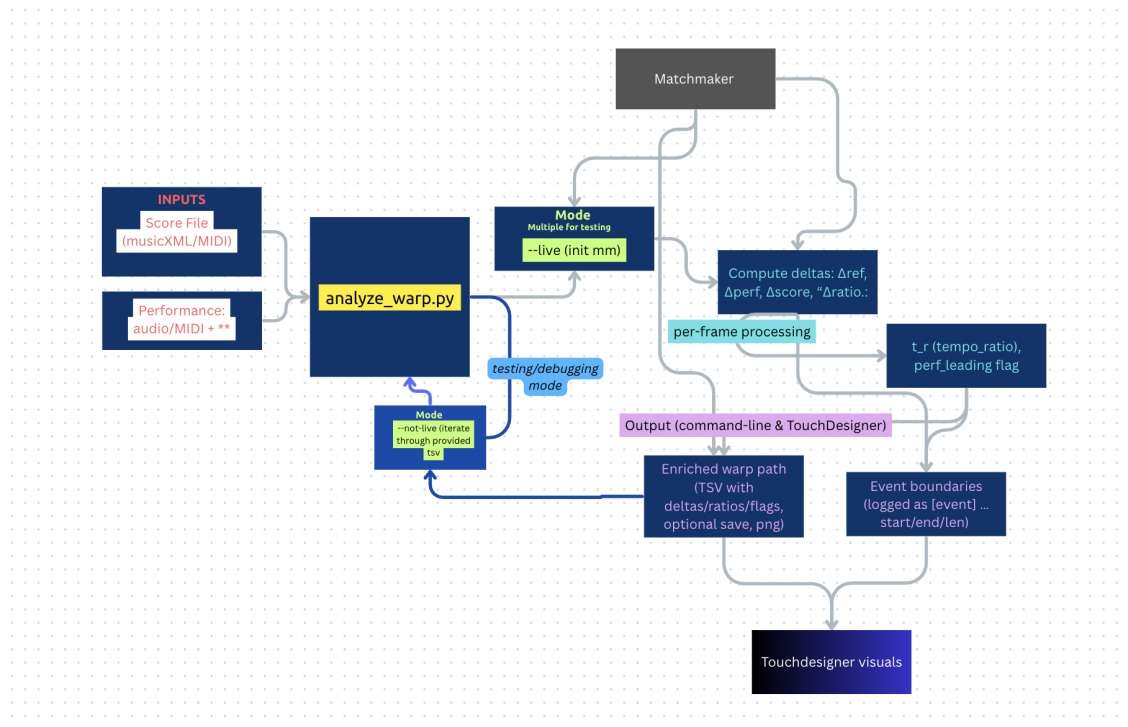


Figure 3.1: Diagram of `analyze_warp.py` as a harness around Matchmaker: score/performance inputs flow into live or TSV replay modes, per-frame deltas and tempo flags are computed, enriched warp-path and event boundaries are sent to console/OSC along with terminal output and a `results` folder, and TouchDesigner visualizes the results.

3.4 TouchDesigner / OSC in `Analyze_warp.py`

`analyze_warp.py` can mirror its live alignment state to TouchDesigner via OSC. Integrated within the CLI, there are parse commands `--osc`, `--osc-host` and `osc-port`; when `osc` is set, the script instantiates a `SimpleUDPClient` and pushes per-frame updates and event markers listed above.

Per-frame stream: on each frame, the script sends `/warp` with `[ref_idx, input_idx, delta_ratio, tempo_ratio, dt_score, delta_ref, delta_perf, perf_leading_flag]`. This gives TouchDesigner a continuous view of where the follower thinks the performer is in the score, how fast it's moving relative to the score, and whether the performer is instantaneously leading (`perf_leading_flag`). Contiguous runs are flagged and emitted as soon as they close. `/event` messages carry `["perf_leading"...]` or `["score_jump", start_frame, end_frame, length]` letting TouchDesigner trigger visuals on performance-leading stretches or score jumps.

A separate logic to account for a `performance_lagging` feature has also been implemented in its latest version of `analyze_warp.py`, but not shown to work in video demonstrations.

Within an implemented TouchDesigner project, an OSC CHOP (Channel Operators; see appendix A for more information on CHOPs) receives the 8 channel payload listed above. How one chooses to interpret this data verges more into interpretational and artistic discourse (see 4).

Within this project file included in this projects deliverables, a `select2` CHOP pulls the first two channels (`ref_idx` and `perf_idx`) for positional use. A `math2` CHOP scales and offsets these indices and essentially normalizes a UI range before passing down into TOPs (Texture Operators) for direct translation into graphics.

`select1` pulls the middle payload (`dt_score` and `delta_ref`). `math1` rescales that signal, `lag1` smooths it to avoid jitter, and `null1` eliminates outliers for neater TOP wiring.

`noise1` begins a TOP feedback loop (`feedback1` which feeds into `level1` through a keyboard-controlled tap, set by the `keyboardin1` DAT. The signal is sent from `level1` into a `slope1`, `blur1`, and `noise2` TOP that adds motion and softens feedback before re-entering this loop. This builds the evolving base texture so that it is continuous in motion as each new event occurs.

Another chain that takes from the first motion chain and is fed into a displacement map TOP `displace1` and `edge1` TOP. The two inputs are taken into `fbend1` and another edge TOP `edge2` before normalization. This is the main edge-bending mechanism that reacts with more drastic angles across shapes as bigger changes in data occur.

Another fan-out occurs with the feeding of `edge2` into a normal map TOP `coloradj1` that effectively remaps color. These give the incrementally darker shifts.

The `noise3` TOP assists in filling in the generated overlaying shapes smoothly, while the `comp1` merges the color-adjusted edges with the `coloradj1` noise overlay that operates on a constant `bg` output TOP that changes over each run.

4 Results

This results section offers a glimpse into the process modifying my methodology and arriving at the current version of `analyze_warp.py`.

4.1 Reconciling Playback Timing With Frame Output: Discovering What “Frame Rate” Actually Meant

One of the earliest difficulties in interpreting Matchmaker’s output came from a mismatch between what I assumed “frame rate” meant and what the system actually produced. During initial tests using the “audio + performance file” playback mode, changing the `--frame-rate` parameter produced a puzzling effect: the frame-by-frame alignment output would finish long before the audio playback ended. At first this made it seem as if Matchmaker were running “faster than real time,” or that the frame rate parameter directly controlled playback speed.

This misunderstanding forced a re-examination of what a “frame” represents inside Matchmaker. The CLI presents frame rate in familiar terms—frames per second—but the analysis mode does not synchronize frame emission to wall-clock playback. Instead, Matchmaker emits frames as fast as the alignment can be computed, using frame boundaries derived from hop length, not real-time pacing. This meant then that a higher declared frame rate (i.e., a shorter hop length) simply creates more frames to compute, and the program processes and prints them as quickly as Python can execute, completely decoupled from the audio timeline. I would discover that the default frame rates of 30 frames/second would work (i.e. have an audio playback that is as close to the frame by frame output) only because of the hard-coded hop length that I couldn’t consider until recently. Even more confusing was that this behavior persisted in scenarios that should have resembled live following. When feeding Matchmaker audio incrementally—or simulating real-time chunking—the output frames would still behave in non-intuitive ways: in several tests the score index (`ref_idx`) accelerated far ahead, sometimes leaping toward the end of the piece while the performance index (`perf_idx`) was still in the middle. The system would then “wait” for the performer to catch up, holding its predicted position until subsequent audio frames brought (`perf_idx`) into alignment with the warped score position. This effect was not merely a quirk of offline processing: it surfaced because the alignment algorithm is allowed to jump to high-confidence anchors anywhere in the score as soon as their features are detected, even if the performer has not yet reached them. Even in pseudo-live mode, Matchmaker’s warp computation is governed by feature salience and global cost minimization, not by a requirement that score time increase monotonically with performance time. Importantly, this premature jumping behavior only appeared in

experiments where I pushed the declared frame rate well beyond Matchmaker’s default without yet modifying hop length or rebuilding the downstream audio processors. In that misconfigured state, the system reported a much denser frame rate than it was actually using internally. The result was a logical mismatch because the warp computation still operated on the coarser, default hop size, but the printed frame indices implied a much finer temporal resolution. Under these conditions did matchmaker appear to teleport and sprint far ahead in the score. Once the hop length was correctly recomputed—so that the true temporal resolution matched the requested frame rate—this extreme forward-jumping behavior diminished and became interpretable within the normal clamp-and-reanchor pattern described later. This clarified that this problem was not inherent to Matchmaker’s alignment algorithm, but rather a consequence of inconsistent temporal settings set in how `analyze_warp.py` worked.

Being able to identify the computing matrix and feature extraction process from each run was extremely valuable in understanding how sampling worked in Matchmaker (see above Figure 4.1). Seeing the full similarity surface laid out made it possible to connect individual frame outputs to their underlying feature computations: which spectral regions contributed most strongly to alignment, how dense or sparse the extracted features actually were at different frame rates, and where ambiguities emerged in the score–performance comparison. Patterns that appeared abstract in the per-frame logs—such as clamps, sudden score jumps, or stretches of confident diagonal following—became legible once mapped onto the matrix, revealing how the warp path navigates between high-similarity ridges and low-confidence valleys. This visualization also clarified the effect of earlier debugging missteps, such as pushing the frame rate without properly adjusting hop length: the resulting discontinuities in the matrix directly mirrored the numerical inconsistencies seen in the warp path. Overall, the ability to inspect the matrix grounded the alignment results in a concrete computational geometry and made the behavior of the follower interpretable rather than opaque.

The reader will scan through the section headers at the first pass. Subsection headings help organizations and help the reader find the parts of interest.

Make section headings as specific and information-rich as possible. Make sure to interpret the results. Summarize the collection of experimental results as clearly and as economically as possible.

4.2 Interpreting Early Behavior in `tempo_ratio` and `delta_ratio`

A second set of challenges arose from interpreting the tempo-side metrics. Early tests showed `tempo_ratio` producing unintuitive values—mostly nan and 1, and only sometimes simple rational values such as 0.3333 or 0.7333. It was as if the metric could only exist in three states—undefined, perfectly aligned, or something arbitrarily “mathematical”—without conveying any interpretable musical meaning.

Only after close inspection of the warp logs did the reason become clear. `tempo_ratio` is only defined when the score advances. During clamped intervals, Matchmaker freezes Δ_{ref} at zero while Δ_{perf} continues moving forward, which forces `tempo_ratio` to be undefined (and thus nan). When Δ_{ref} finally increases, it often increases by exactly one

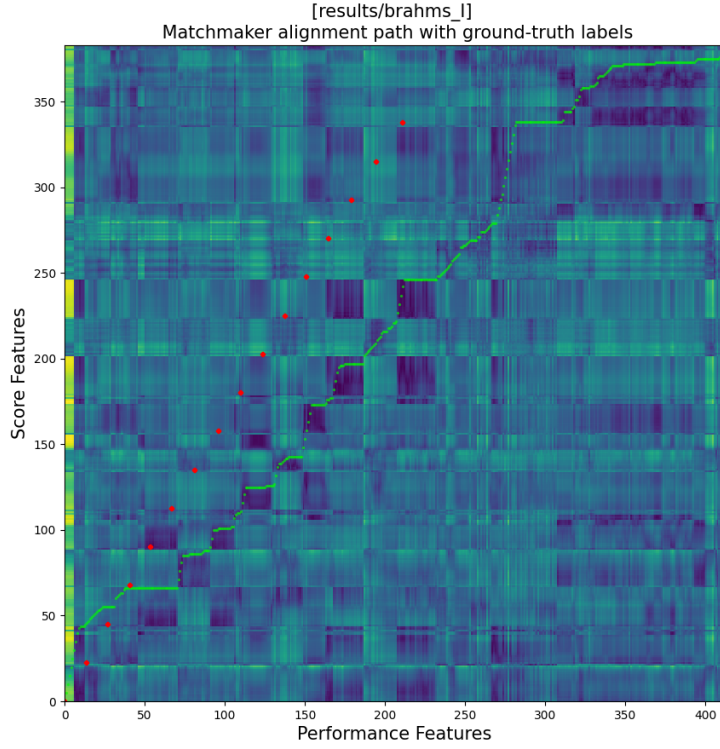


Figure 4.1: The background matrix encodes feature similarity between performance audio (horizontal axis) and score features (vertical axis) with a running of Brahms.I.wav (see Appendix A for further details on testing files). The neon yellow–green line is Matchmaker’s computed warp path; red dots represent the human-annotated ground-truth alignments. Diagonal regions indicate confident following, while horizontal plateaus reflect periods where the performer leads the model ($\Delta_{\text{ref}} = 0$, $\Delta_{\text{perf}} > 0$), and vertical jumps correspond to score re-anchoring events (“score_jumps”). The clear correspondence between warp geometry and event-summary behavior validates that the per-frame metrics used in TouchDesigner faithfully reflect these underlying alignment dynamics.

score frame—producing a `tempo_ratio` of $(\Delta_{\text{perf}}/\Delta_{\text{ref}} = \Delta_{\text{perf}}/1)$, which early on tended to be 1 because the follower was stepping forward in lockstep. The few rational values emerged when small variations in Δ_{perf} occurred, but the prevailing pattern of `nan` $\rightarrow$ 1 $\rightarrow$ `nan` was not a numerical artifact: it reflected the fact that the follower was not yet distinguishing tempo differences because the score index was not moving in a musically meaningful way. To preserve continuity in output during these freezes, I introduced `delta_ratio`, which acts as a stand-in for `tempo_ratio` whenever $\Delta_{\text{ref}} = 0$. I also introduced `grad`, the per-frame ratio of performance advance to score advance. It is usually $\Delta_{\text{perf}} / \Delta_{\text{ref}}$ when the score moves, but if the score is clamped, it falls back to Δ_{perf} (or `nan` at the very first frame). Values near 1 meant that the performance and score step were in sync; > 1 the performance moves faster than the score; and < 1 it is lagging behind the score. But this brought its own complications when `delta_ratio` could spike dramatically when the performer moved ahead of the score model, and it could stagnate when they remained aligned. These edge cases were not errors but consequences of Matchmaker’s clamp–then–jump behavior. The difficulty, then, was not in computing these ratios but in assigning meaningful interpretation. A `tempo_ratio` greater than 1 does not necessarily mean the performer “played faster”; instead, it describes the follower’s instantaneous perception of how performance pacing compares to score pacing, given the geometry of the current warp path. Likewise, the prolonged stretches of `tempo_ratio` = 1 in early tests were not signs of stability but of insufficient frame advancement on the score side.

The subtlety became especially important when visualizing the metrics in TouchDesigner. Naïve thresholding on raw ratios caused severe flicker in TouchDesigner; states oscillated too rapidly to be perceptually meaningful. Adding a form of “hysteresis” (more on its implementation below) through simple lag filtering in the Math/Lag CHOP chains and revealed more musical behavior beyond what was implemented in `analyze_warp.py`.

Through this debugging, I learned that `tempo_ratio` is less the scalar tempo change estimator I wanted it to be and more a local differential descriptor of warp-path geometry. Without smoothing, contextualization, and an understanding of when Δ_{ref} acutally moves, the metric appears either useless or erratic.

Any labels or uncommon abbreviations need to be explained in the figure or table caption.

4.3 A Qualitative Description of Current TouchDesigner graphics

The TouchDesigner component of this project translated the alignment metrics into a generative visual environment, but its current form reveals both the strengths and limitations of my implementation. The graphics respond to OSC input—through color changes, directional flourishes, and turbulence within the feedback network—but much of the visual output still behaves semi-randomly. The system reacts, but not in a reliably interpretable way. As a result, it often becomes difficult to distinguish which aspects of the visuals are meaningful reflections of the alignment data and which are simply artifacts of the genera-

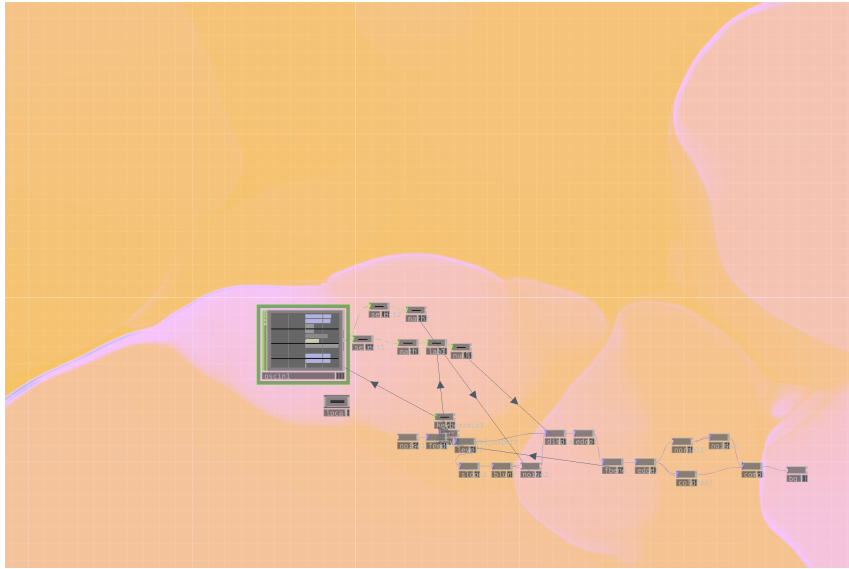




Figure 4.3: TouchDesigner network and feedback surface while Brahms_I.wav is actively running. The screenshot is taken approximately two seconds after pressing the reset key (“1”), which restarts the feedback loop while the audio and alignment stream continue. This frame captures the early evolution of the visualization as OSC signals begin influencing the texture.

the level of a coherent interpretive tool yet. It shows that something is happening, but not clearly what or why. Significant room remains for more intentional mappings between data and visual structure, as well as for integrating accumulative metrics that could provide musical meaning rather than momentary sensation. In this sense, the visualization is more a proof of concept than a final product—an important stage in development, but one that invites substantial refinement in future work.

5 Related Work

This project could not have existed in its submitted form without the amazing work of Park, et al and their development in creating Matchmaker. Strides to not just further efficient and accurate alignment methods in score following are just as important as it is making the work open-source and accessible to the public, and the library has been remarkable in its scope and execution.

Modeling expressive traits in musical playing has been a research topic since around the same time that score following has. Methods involving machine learning and Hidden Markov Models show some overlap in the world of live-time score following, with work from Grindlay & Helmbold (2006) [3] demonstrating how expressive timing, rubato, and phrasing can be captured through statistical models with far more intricate mathematical approach. Their findings underscore that expressive performance cannot be reduced to simple tempo changes; instead, it emerges from complex, frame-by-frame timing relationships. This perspective is closely aligned with the goals of this thesis, which examines how Matchmaker’s per-frame alignment metrics encode similar expressive deviations in real time.

Work done by Madeline Wong (2006) [4] explores similar notions of tempo curves being used to map expressivity in timing. In her work, expressive performance is modeled as a continuously varying tempo trajectory that can be predicted from features of the musical score. Rather than treating timing deviations as noise or as isolated events, Wong frames rubato and expressive shaping as coherent, learnable functions that reflect phrase structure, harmonic tension, and stylistic norms. By training machine-learning models on recorded performances, she demonstrates that tempo can be synthesized automatically in ways that resemble human expressive behavior. This perspective aligns closely with the concerns of the present thesis: both approaches regard expressive timing as a structured signal rather than a by-product of performance variability. However, where Wong assumes a clean alignment between score and performance in order to learn such curves, the work here interrogates the alignment stream itself—examining how systems like Matchmaker infer timing relationships frame by frame and how those inferences can be interpreted or visualized in real time.

Gasser’s thesis [5] offers one of the more detailed treatments of how expressive musical performance can be visualized in ways that foreground musical intention, structure, and phrasing though not live-time to a performance. His visual metaphors translate expressive parameters—such as timing deviations or dynamic shaping—into smoothly interpretable visual forms. However, his system operates primarily on precomputed performance data rather than on live data streams. It is therefore not designed for real-time responsiveness or integration with a score-following backend. In contrast, the present work investigates how raw, rapidly changing alignment metrics from Matchmaker be-

have in real time and how they might be stabilized, interpreted, and mapped to generative graphics. Where Gasser begins with curated expressive features, this project begins with the lower-level alignment signal itself, making the visualization challenge fundamentally different: the task is not to depict known expressive structure, but to shape and interpret the noisy, per-frame processes through which a follower discovers that structure during performance.

6 Conclusions

This project began with several guiding research questions: How can we meaningfully characterize expressive timing in performance using computational score following? What metrics best capture the nuanced ways performers deviate from notated time? And how might visual systems make these deviations legible in real time? Over the course of this work, these questions proved far more intricate than their initial formulations suggested, revealing both the promise and the complexity of aligning algorithmic analysis with musical interpretation.

A central contribution of this thesis is the clarification and operationalization of key alignment metrics—particularly `delta_ratio` and `tempo_ratio`, and the event-state logic surrounding clamping, leading, and score jumps. Although straightforward in definition, these metrics turned out to behave in highly nontrivial ways, shaped by Matchmaker’s dynamic programming structure, its tendency to freeze and re-anchor, and the interaction between hop length, frame rate, and temporal sampling. Flushing out these behaviors was essential: without understanding the geometry of the warp path and the circumstances under which Δ_{ref} and Δ_{perf} move, expressive timing analysis risks becoming either misleadingly simplistic or unintentionally opaque. The implementation work in Python therefore served not just as both an engineering and interpretive task—creating the conceptual scaffolding necessary for any downstream use of these metrics. At the same time, the project’s limitations reveal the real constraints under which this work was completed. Time was a significant factor, particularly given the need to learn Python from the ground up and to develop a working understanding of the Matchmaker library—an advanced, sparsely documented system whose internal structure required substantial reverse-engineering. These constraints meant that many design choices, especially in the TouchDesigner visualization, were necessarily exploratory rather than exhaustive. The current system is reactive and aesthetically engaging but lacks accumulative structure posed by my advisor, Scott Petersen: it visualizes instantaneous disturbances rather than long-range musical behavior. This shortcoming is less a failure of concept than a reflection of the project’s scope; a more mature system would incorporate temporal memory, richer mappings, and more intentional interpretive models in the future. More broadly, this thesis demonstrates that expressive timing analysis depends as much on the interpretation and design of metrics as on alignment algorithms themselves. Refining the computational pipeline—understanding how features are compared, how sampling affects confidence, how events emerge, and how artifacts arise—proved to be necessary groundwork for any meaningful interpretation. The work here represents an initial but substantial step toward a system that both tracks musical performance with technical rigor and communicates its findings intuitively.

By identifying where these metrics fail, where the visualization obscures rather than

clarifies, and where the model’s assumptions introduce distortions, we surface the conditions under which expressive timing can be represented faithfully and in interesting interpretative ways. What emerges is a roadmap for future work on this intersectional study: developing accumulative interpretive layers, refining event detection, experimenting with alternative feature extraction strategies, and designing visualization systems that prioritize temporal structure over randomness. These questions remain open, but they now stand on firmer conceptual footing—built from the careful examination, implementation, and critique of the tools explored in this thesis.

7 Future Work

Further development of this project is planned, motivated both by the natural extensions identified during the present research and by the requirements of the MUSI4996 “Computing and the Arts” component of my senior thesis. Because MUSI4996 entails a performance-oriented realization of the system, the next phase of work will focus on strengthening the stability, interpretability, and artistic flexibility of the alignment and visualization pipeline. The work outlined below will either fall within the MUSI4996 scope or constitute additional development completed over break in preparation for the performance requirement.

First, additional hysteresis and noise-reduction strategies will be implemented within `analyze_warp.py` to stabilize per-frame event detection. Although the current system incorporates minimal smoothing for visualization, more robust filtering is necessary for reliable use in performance contexts. Second, I plan to design accumulative structures—both in TouchDesigner and within `analyze_warp.py`—that characterize longer-term trends in the alignment stream. These structures will support the interpretation of performer-leading behavior, clamping frequency, drift tendencies, and other patterns that are not evident from instantaneous metrics alone.

Third, substantial work remains on the TouchDesigner environment itself, including refining the existing visual mappings, reducing stochastic artifacts, supporting multiple visualization “styles,” and configuring the system for different types of repertoire. Extending the system beyond piano performance is another important direction. Matchmaker’s backend will be tested with non-piano instruments to determine the feasibility of real-time alignment in more heterogeneous settings. If these experiments prove successful, the next stage will be to develop a performance with an Chromas Ensemble, the orchestra I have founded and manage, integrating live alignment and visualization across multiple instruments in a comprehensive program showcasing different styles of music juxtaposed with different configurations of TouchDesigner graphics.

If non-piano compatibility proves limited or inconsistent, the performance component for MUSI4996 will instead focus on constructing a comprehensive demonstration showcasing multiple TouchDesigner configurations applied across varied piano repertoire, including my own playing. This fallback path still enables a full artistic realization of the system while maintaining technical fidelity to Matchmaker’s current capabilities.

Together, these directions outline a clear trajectory for extending the technical, analytical, and performative dimensions of the project, aligning the computational work completed here with the performance-focused expectations of MUSI4996.

Beyond the scope of MUSI4996, I anticipate continuing this line of work as part of my long-term development as a pianist. Boris Berman describes the cultivation of an “objective ear” as an essential skill for performers, referring to the capacity to hear one’s own

playing with the clarity and neutrality of an outside observer [9]. Tools that surface timing behavior, alignment tendencies, and patterns of phrasing in real time may contribute to this kind of self-evaluation by making aspects of performance practice visible or measurable in ways that ordinary listening cannot. Extending the system into a practice-oriented framework—one capable of tracking expressive deviations during rehearsal, comparing different interpretive attempts, or highlighting patterns that recur unconsciously—would allow the technology developed here to function not only as an artistic visualization tool but also as a pedagogical aid. In this sense, the future trajectory of this project lies equally in performance and technology: refining computational techniques that can support a performer’s ongoing effort to hear themselves more truthfully and to develop a more self-aware and intentional interpretive practice.

Bibliography

- [1] Jiyun Park et al. “Matchmaker: An Open-Source Library for Real-Time Piano Score Following and Systematic Evaluation”. In: *Proceedings of the 26th International Society for Music Information Retrieval Conference (ISMIR 2025)*. Daejeon, South Korea, 2025.
- [2] Arshia Cont. “A Coupled Duration–Focus Architecture for Real-Time Music Tracking”. In: *Computer Music Journal* 33.1 (2010), pp. 76–92.
- [3] Graham Grindlay and David P. Helmbold. “Modeling, Analyzing, and Synthesizing Expressive Piano Performance with Perceptron Learning”. In: *Proceedings of the 7th International Conference on Music Information Retrieval (ISMIR)*. 2006, pp. 356–361.
- [4] Madeline Wong. “Towards Automatic Generation of Expressive Tempo Curves”. MA thesis. MIT, 2006, pp. 1–4.
- [5] Martin Gasser. “Interactive Visualization of Expressive Piano Performance”. MA thesis. Johannes Kepler University Linz, 2005.
- [6] Eva Istók et al. “Expressive Timing Facilitates the Neural Processing of Phrase Boundaries in Music: Evidence from Event-Related Potentials”. In: *PLOS ONE* 8.1 (2013), e55150. DOI: 10.1371/journal.pone.0055150.
- [7] Charles Beatty et al. “The (n)ASAP Dataset for Expressive Piano Performance Analysis and Alignment”. In: *Transactions of the International Society for Music Information Retrieval* 6.1 (2023), pp. 1–18.
- [8] Roger B. Dannenberg. “An On-Line Algorithm for Real-Time Accompaniment”. In: *Proceedings of the 1984 International Computer Music Conference*. ICMA, 1984, pp. 193–198.
- [9] Boris Berman. *Notes from the Pianist’s Bench*. 1st ed. Discussion of the “objective ear” appears in early chapters. New Haven, CT: Yale University Press, 2000.

A Appendix Title

A.1 Soundfont Details + Additional Assets

This project required the use of some sort of soundfont in order to generate a syntehtic rendering of .musicxml scores. Soundfonts are generally in the public domain, but I would like to credit user "Shiroi Channel" for sharing the Steinway B (1956) soundfont on Polyphone. The soundfont used can be found at: [This Link](#).

Additional assets. I added a Steinway B soundfont for rendering (`tests/Steinway_B_soundfont.sf2`) and a custom Brahms Op. 3 No. 5 excerpt (first seven bars) with both score and audio: the MusicXML score `matchmaker/assets/Brahms_I_XML.mxl`, the performance recording `matchmaker/assets/Brahms_I_test_.wav`, and the MIDI sketch used earlier for MIDI-based tests `tests/brahms_pc_.mid`. Alignment artifacts from these runs (warp-path TSVs, evaluation summaries) live under `results/` with filenames prefixed by the run name (e.g., `brahms_pc_*`). All of these are user-provided and not part of the original (n)ASAP dataset or testing suite.

A.2 Matchmaker Sound Features

Matchmaker performs score following by comparing multidimensional audio features extracted from both the performance signal and a synthesized score representation. These features include chroma vectors for pitch content, MFCCs and other spectral descriptors for timbral information, and onset-strength measures for detecting note attacks. Combined, these features provide the alignment algorithm with harmonic, timbral, and temporal cues that allow it to track a performer through expressive timing variation.

When feature types are toggled in `analyze_warp.py`, Matchmaker rebuilds the underlying audio processors using only the selected subset of features. This changes the dimensionality of the feature vectors that populate the cost matrix and can alter which musical cues dominate the alignment. For example, enabling only chroma features makes the follower rely primarily on harmonic content, while including MFCCs or onset-strength features introduces timbral and attack information that may improve tracking in dense or percussive passages. Each toggle therefore reconfigures the evidence that Matchmaker uses for alignment, allowing users to experiment with how different combinations of features affect clamping, re-anchoring, and overall alignment behavior. Future work will definitely be focused on using these features when entering the non-piano experimental phase of testing.

A.3 TouchDesigner

TouchDesigner provides a node-based environment for real-time graphics, and the visualization in this project is built primarily from two operator families: CHOPs (Channel Operators) and TOPs (Texture Operators). CHOPs process numerical signals, while TOPs generate and transform images. Matchmaker’s alignment stream is received as OSC data, transformed through CHOP networks, and then drives a chain of TOPs that produce the evolving visual texture.

The OSC CHOP serves as the entry point for alignment data. To enable communication on the same machine, the OSC CHOP is configured in “Network UDP In” mode with the port number matching the one specified in `analyze_warp.py` via `--osc-port`. This ensures that each per-frame message sent by the script (e.g., `/warp`) is mapped to channel values inside TouchDesigner. Once received, CHOP nodes such as `select`, `math`, and `lag` reshape, scale, and smooth the incoming values, ensuring stable control signals for the graphics pipeline.

TOP operators use these processed signals to modulate the evolving feedback surface. A `noise` TOP initiates the motion field, which is routed through a feedback loop consisting of `feedback`, `level`, and `blur` TOPs. Additional TOPs such as `displace`, `edge`, and `color` apply geometric warping and color shifts whose intensities depend on the CHOP-driven parameters. This architecture allows numerical alignment metrics from Matchmaker—such as `dt_score` or `delta_ref`—to influence the texture in real time, even though the core motion remains generative.

Together, the CHOP and TOP networks translate raw OSC data into a responsive visual field that reflects metric fluctuations while maintaining continuous motion through feedback dynamics.

For further details and demonstrations with touchdesigner, view this drive media folder: Media Demonstrations